



SE-Agent: Automated Trust in AI-Generated Code

An AI-Powered Software Engineering Assistant
with Multi-Dimensional LLM Output Evaluation

Zain Abbas, Airaf Siddiqui, Adil Shahzad
Università degli Studi di Napoli Federico II — AI System Engineering
Project Report — Naples, 2026

The Double-Edged Sword of Generative Code

Large Language Models (LLMs) like Claude and GPT-4 have revolutionized productivity, but production adoption is stalled by three critical trust barriers.



Hallucination (The Integrity Risk)

LLMs frequently invent non-existent APIs, import fake modules, or fabricate function signatures, leading to runtime failures.



Security Vulnerabilities (The Safety Risk)

Without guardrails, models generate code susceptible to SQL injection, command injection, and hardcoded credentials.



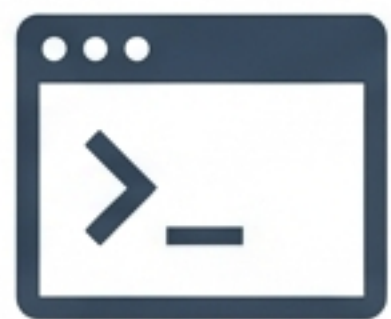
Robustness (The Quality Risk)

Code that "looks" correct often fails on edge cases, null values, or unexpected data formats.

**We need a system that moves beyond "Can it write code?"
to "Can we trust the code it writes?"**

SE-Agent: A Modular Assistant for the Full SDLC

SE-Agent is designed as a modular system built on Anthropic's Claude API. It moves beyond simple chat interfaces to support five distinct engineering capabilities.



Code Generation

Multi-language support with context-aware generation.



Test Generation

Automated synthesis of pytest/unittest cases with edge case targeting.



Code Review

Security-focused analysis with severity classification.



Requirements

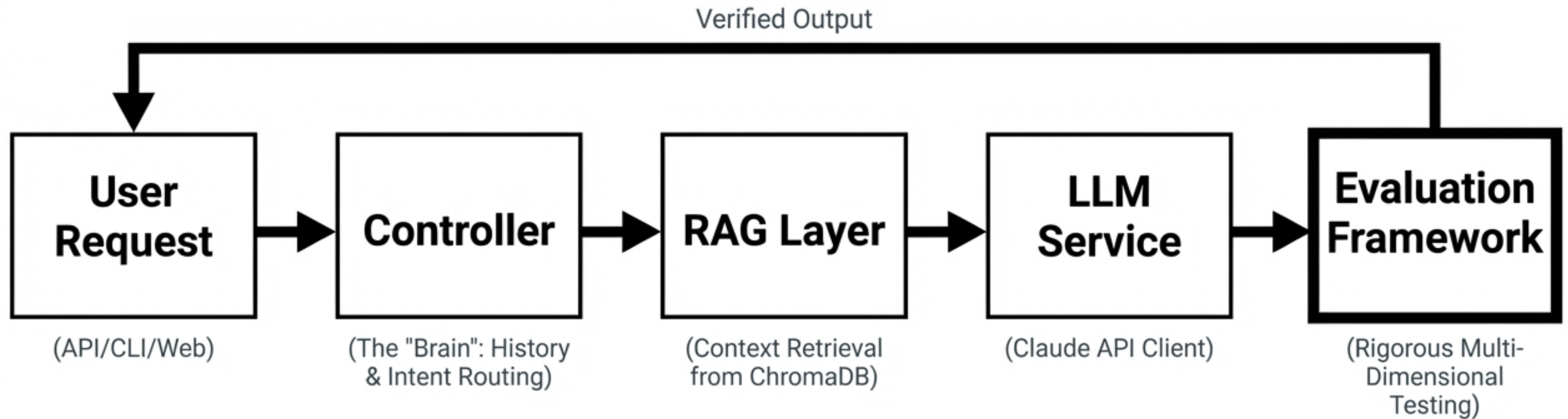
Transforming natural language into user stories and acceptance criteria.



Documentation

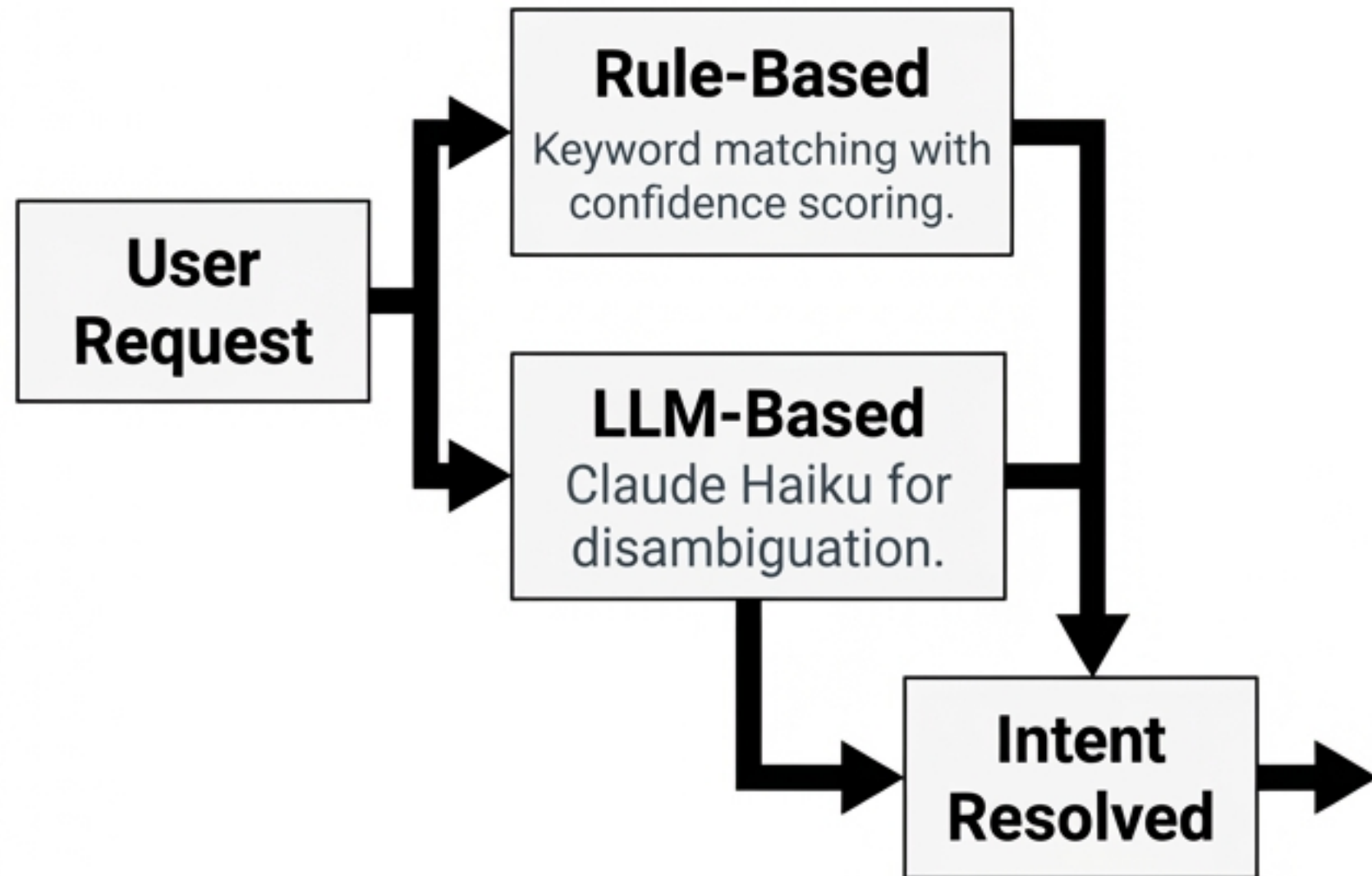
Generating docstrings, API references, and READMEs.

System Architecture & Logic Flow

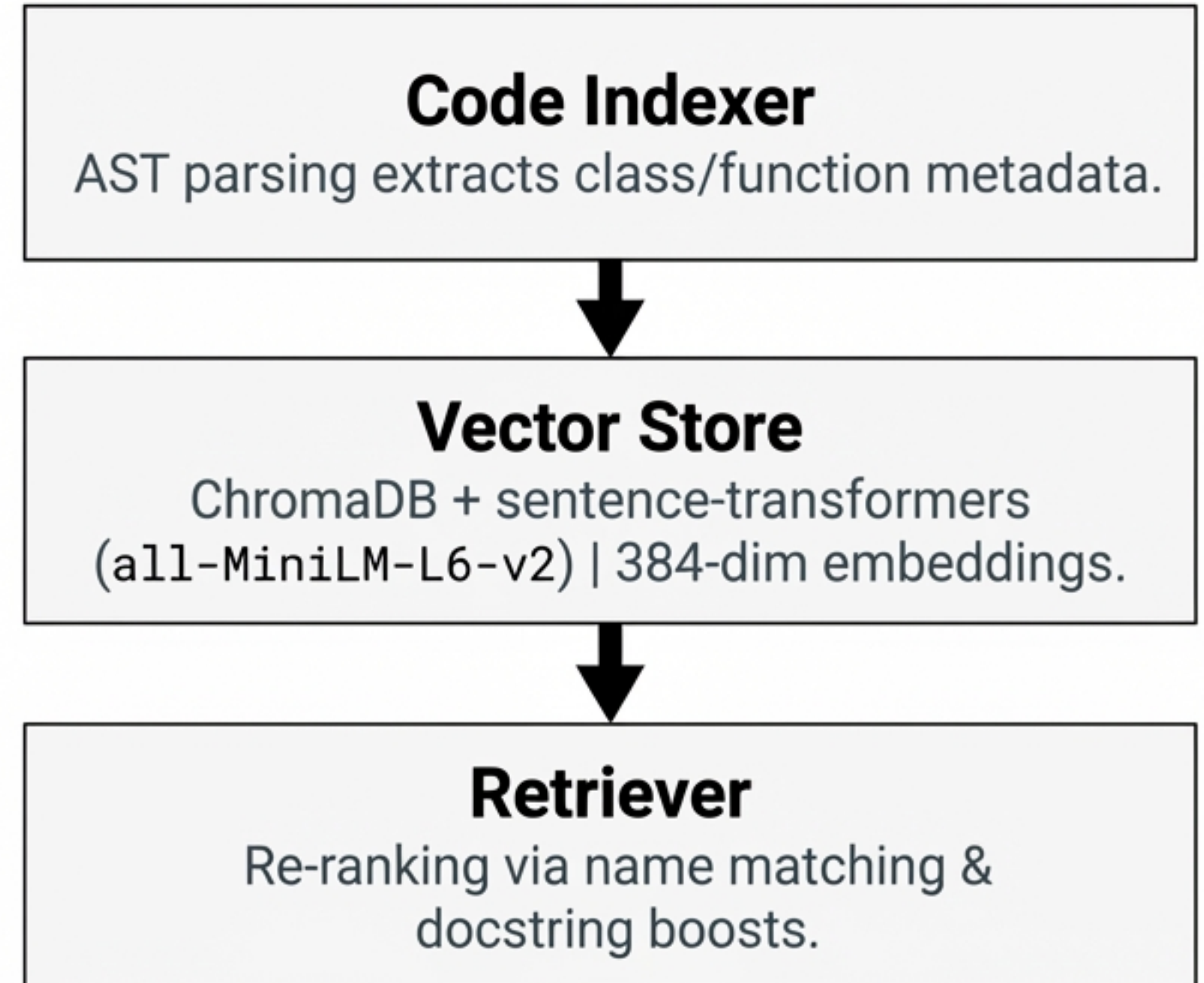


Intelligent Routing & Context Awareness

Intent Routing (Hybrid Classification)



The RAG System (Context Engine)



The Core Innovation: 4-Dimensional Evaluation

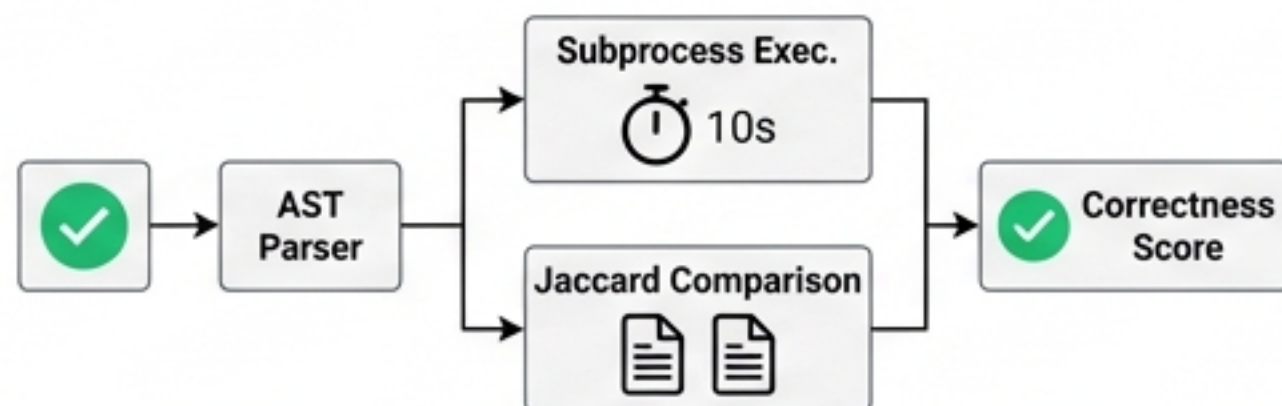
We assess generated code across four orthogonal quality dimensions, applying a weighted aggregation formula to determine a final “Pass/Fail” score.

Correctness	Safety
Weight: 0.35 Threshold: 0.7 Does the code produce expected outputs?	Weight: 0.25 Threshold: 0.7 Is the code free from vulnerabilities?
Robustness	Hallucination
Weight: 0.20 Threshold: 0.6 Does it handle edge cases and inputs gracefully?	Weight: 0.20 Threshold: 0.7 Are the APIs and imports real?

Evaluating Functionality: Correctness & Robustness

Dimension 1: Correctness

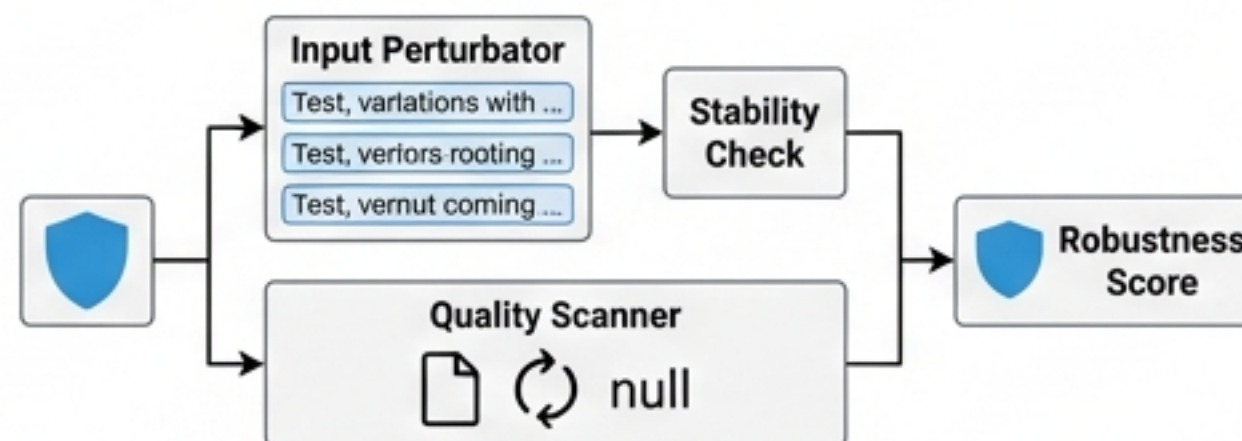
- ✓ **Syntax Validation:** AST parsing to catch syntax errors immediately.
- ✓ **Execution Testing:** Subprocess execution with 10-second timeout.
- ✓ **Semantic Similarity:** Token-based Jaccard similarity against reference implementations.



```
if score < threshold: return FAIL
```

Dimension 2: Robustness

- 🛡️ **Input Perturbation:** Testing response stability under prompt variations (phrasing/spacing).
- 💪 **Quality Metrics:** Automated detection of empty outputs, repetition, or null-handling failures.



Evaluating Trust: Safety & Hallucination

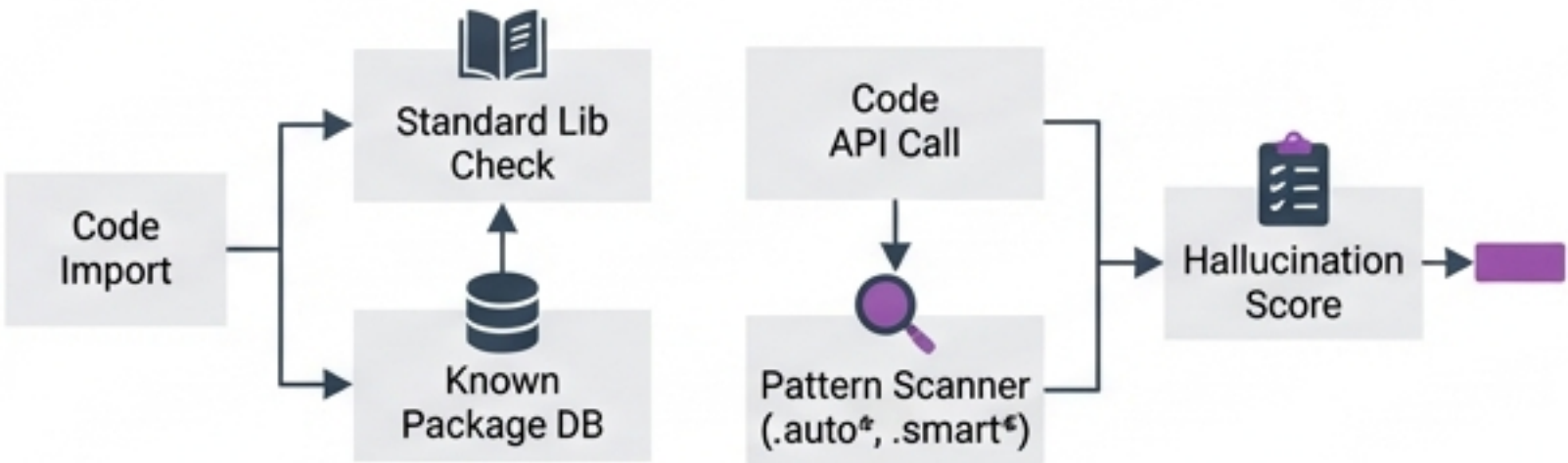
Dimension 3: Safety

-  **Tools:** Integration with Bandit (static security analyzer).
-  **Pattern Matching:** Regex detection for 12 vulnerability categories (SQL injection, XSS, hardcoded secrets).
-  **Scoring:** Severity-based penalties (Critical -0.4 to Low -0.1).

Bandit Report Output	
Issue:	Hardcoded Password
Severity:	High 
Confidence:	High
Location:	line 24
<pre>password = "admin123"</pre>	

Dimension 4: Hallucination Detection

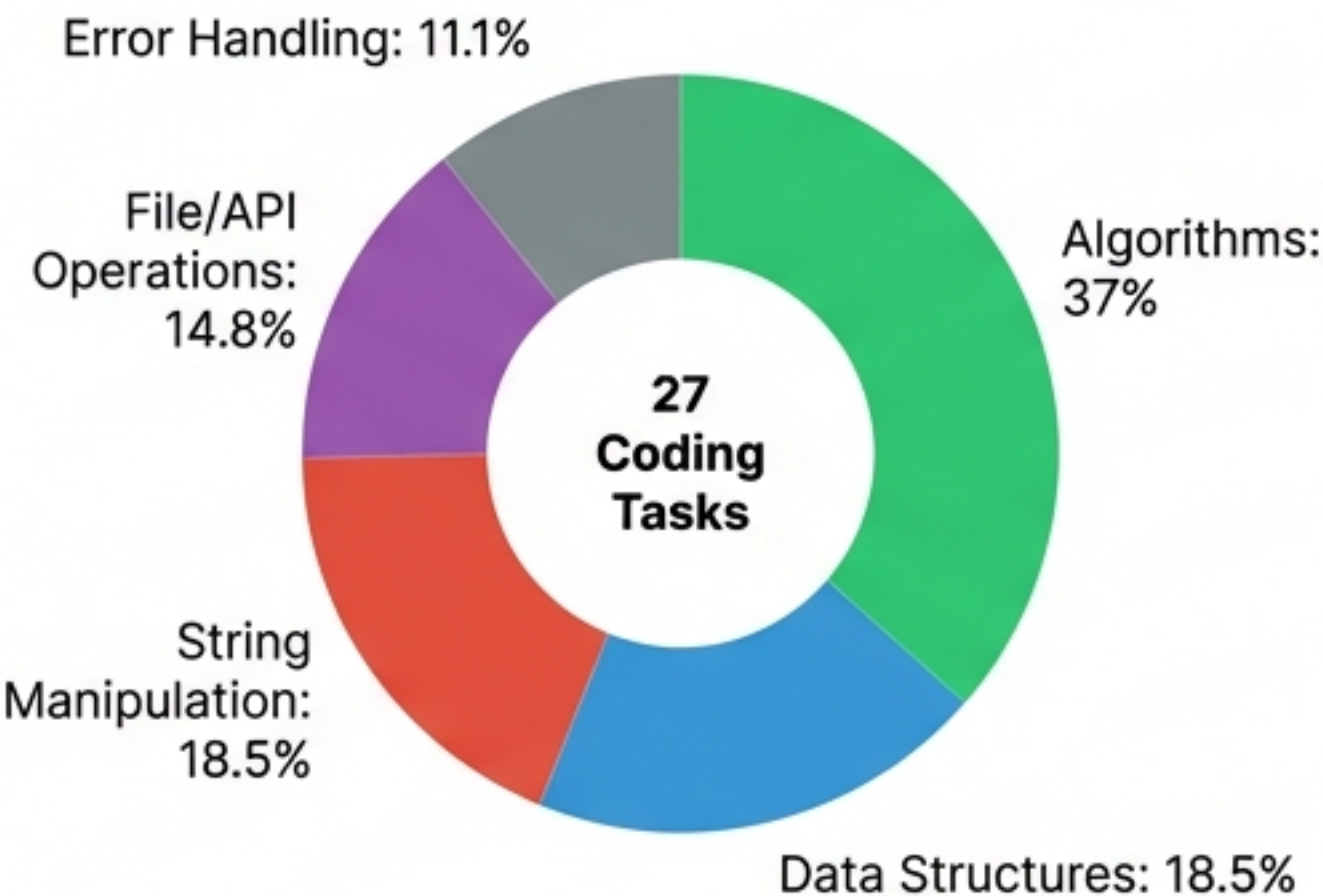
-  **Import Validation:** Verifies imports against Python standard library and known packages.
-  **Fake API Detection:** Scans for suspicious patterns (e.g., .auto*, .smart*) and invalid builtin signatures.



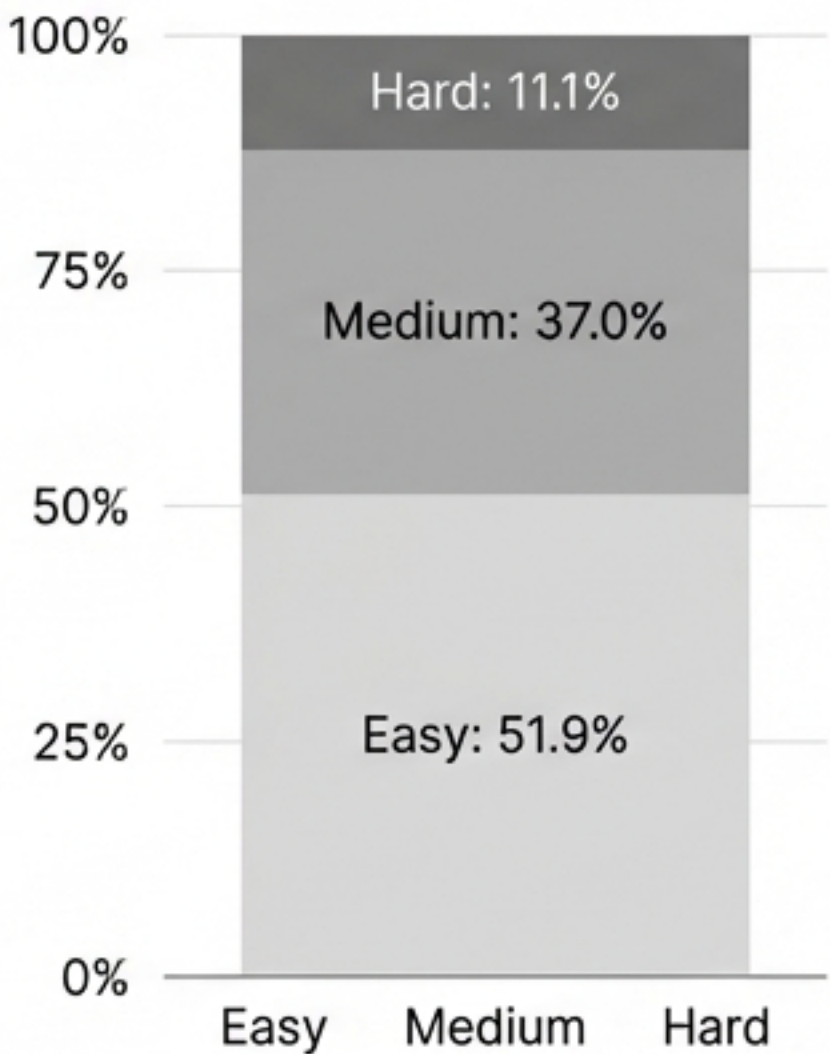
Methodology & Benchmark Dataset

The system was stress-tested against a custom benchmark of 27 coding tasks.

Dataset Composition



Difficulty Distribution



Experiment Configuration

Model:	claude-sonnet-4-20250514
Temperature:	0.7
Max Tokens:	2048

Overall Performance Results

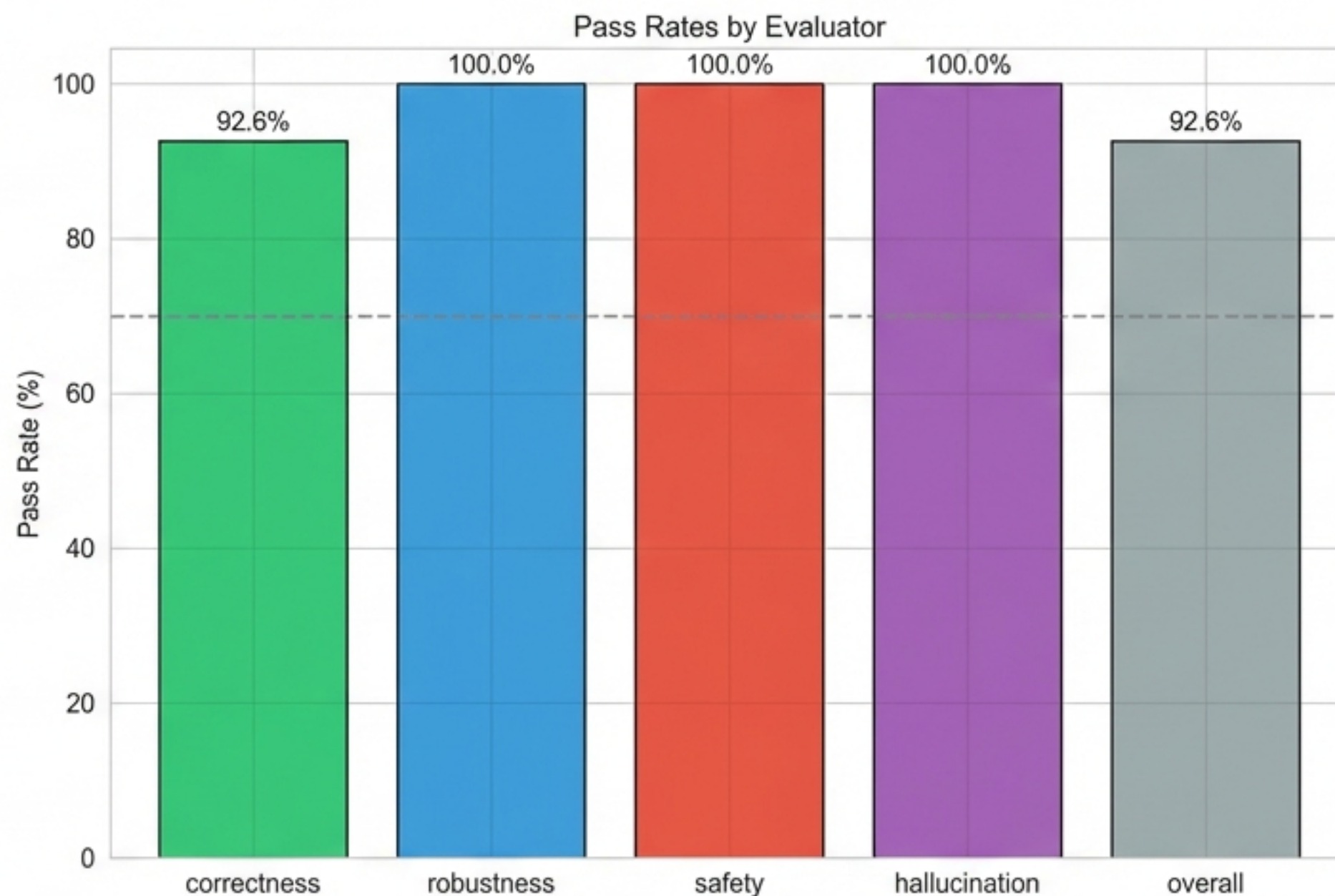
92.59%

Overall Pass Rate (25/27 Tasks)

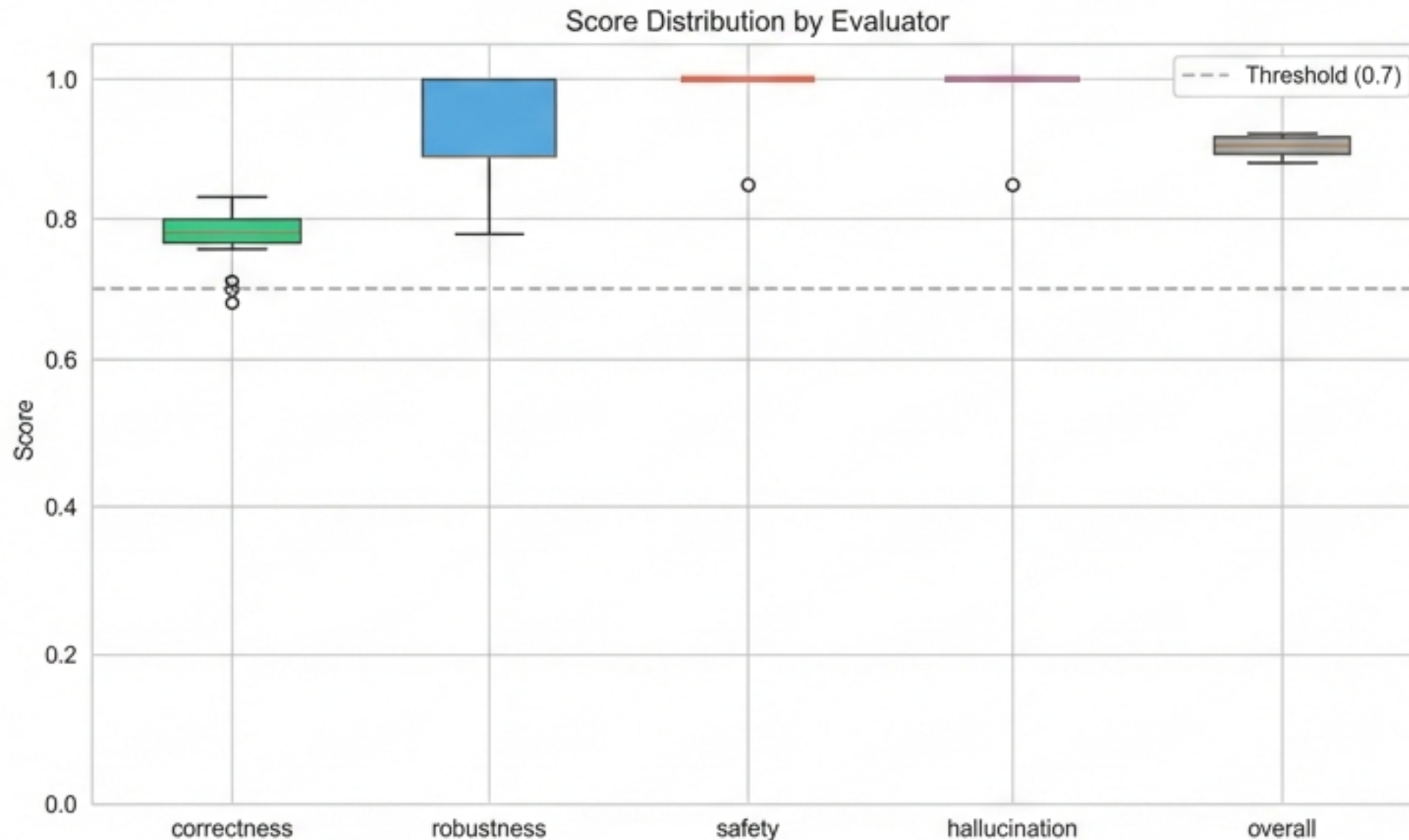
0.905

Average Weighted Score

The system demonstrates high reliability, with the vast majority of generated code meeting the strict multi-dimensional thresholds.



Excellence in Safety & Reality Grounding



- **99.4% Safety Score**

Zero critical vulnerabilities found. The system successfully avoids SQL injection and command injection patterns.

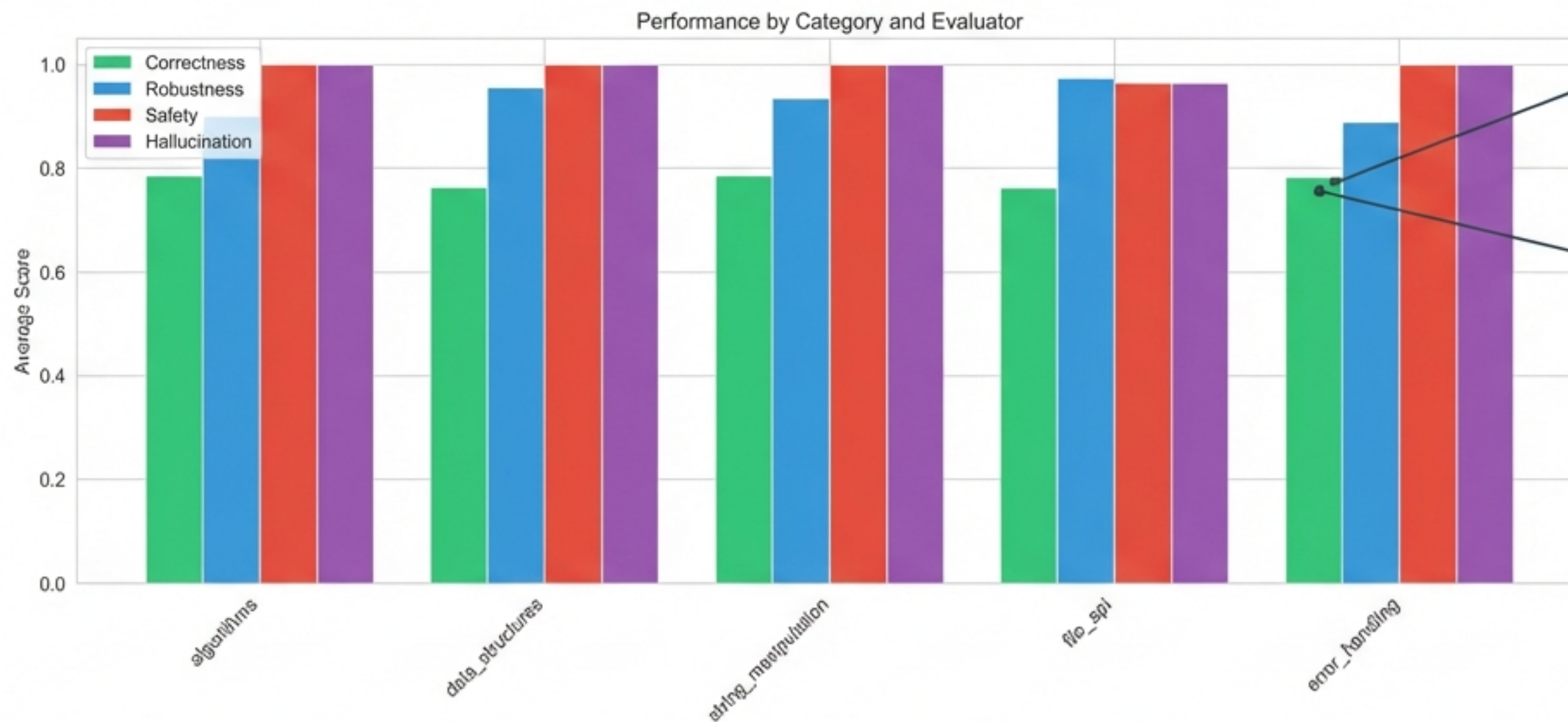
- **99.4% Hallucination Score**

Code is authentic. The system rarely generates fabricated APIs, ensuring maintainability.

- **Correlation Insight**

Strong positive correlation between Safety and Hallucination scores, confirming that 'real' code tends to be 'safe' code.

The Challenge of Correctness



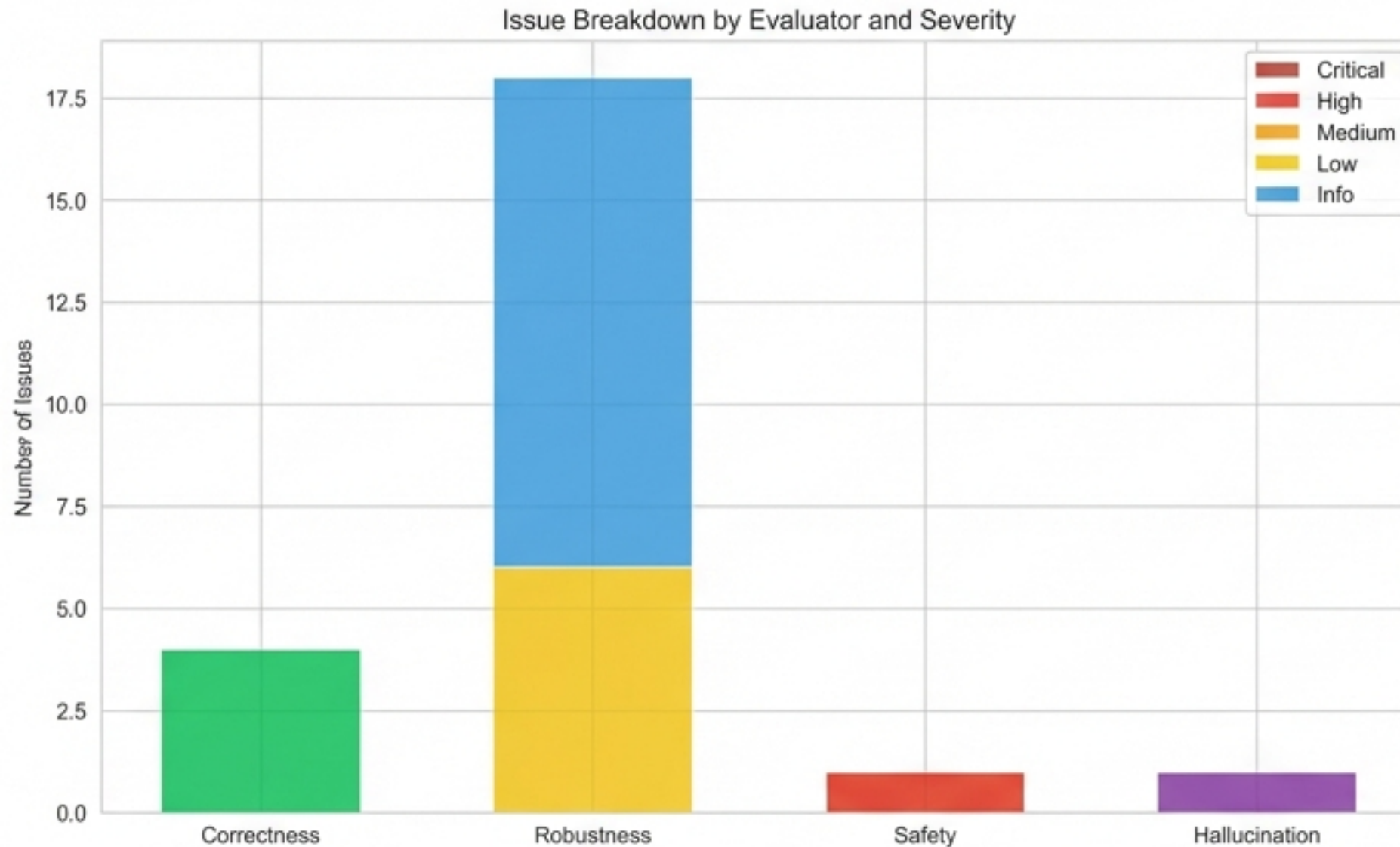
Insight 1: Correctness Score (77.8%) lags behind safety due to test case assertion complexities.

Insight 2: 'Error Handling' tasks showed lower correctness scores due to the complexity of edge case logic.

Insight 3: Hard tasks showed higher variance in correctness, though performance remained relatively stable overall.

Issue Analysis: Informational vs. Critical

75% of identified issues were Informational or Low Severity.



Breakdown Data:

- Missing Error Handling: 12 (Informational)
- Missing Null Checks: 6 (Low Severity)
- Dangerous Imports: 1 (Medium Severity)
- Suspicious API: 1 (Medium Severity)

The system defaults to 'safe but sometimes incomplete' rather than 'functional but dangerous'.

Conclusion & Future Roadmap

Summary of Impact

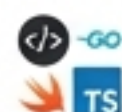
SE-Agent proves that AI-generated code can be trusted when wrapped in a rigorous, multi-dimensional evaluation framework.

Safety: 
99.4%

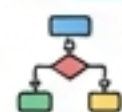
Reliability: 
92.6%
Pass

Hallucination: 
99.4%
Auth.

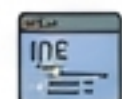
Future Directions



Multi-Language Support: Extending evaluators to Go, Java, and TypeScript.



Pass@k Evaluation: Implementing multiple generations per task to improve **Correctness**.



IDE Integration: Moving from API/CLI to real-time plugin feedback.

SE-Agent: Production-Ready, Secure, Verified.